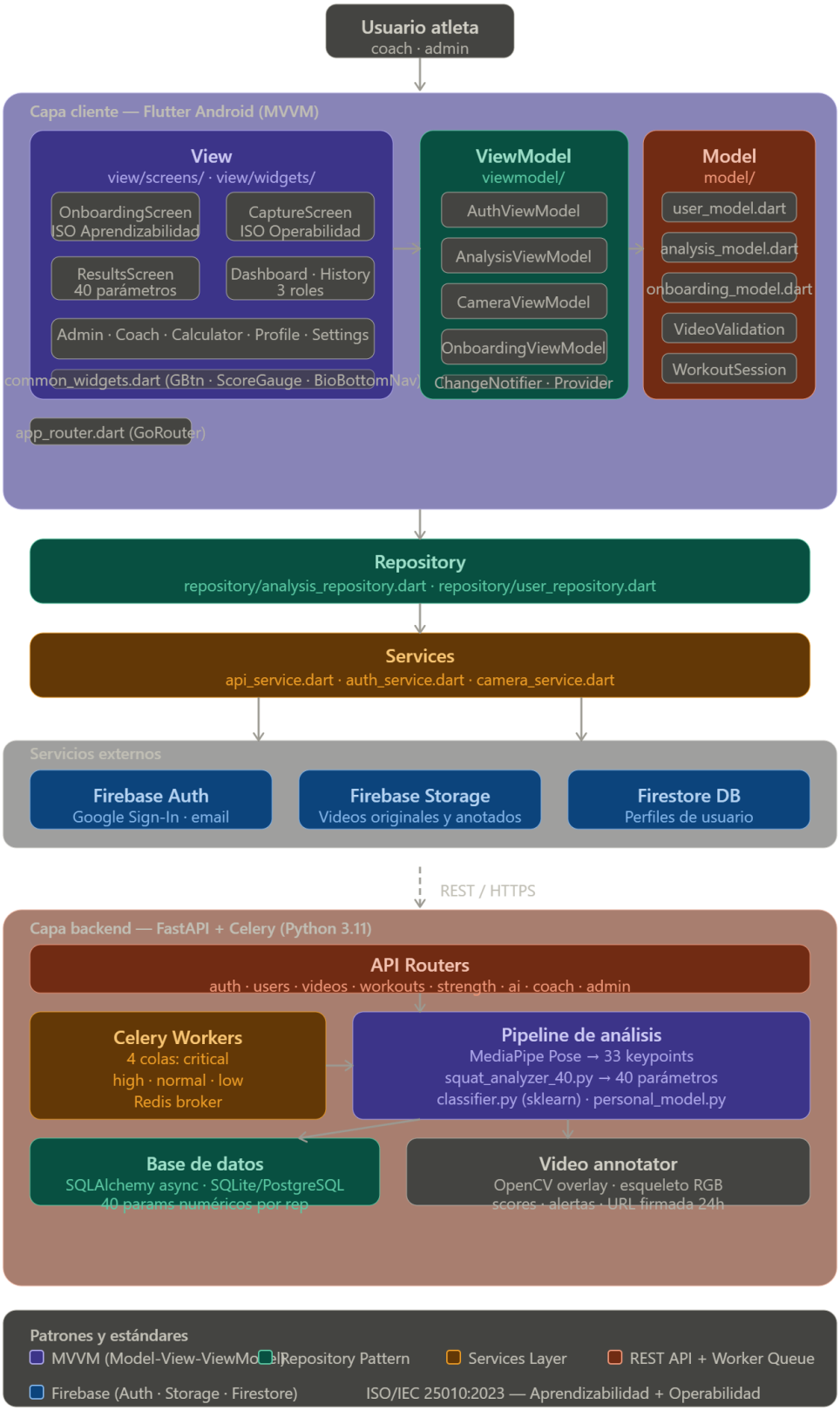
Decisiones arquitectónicas



- Arquitectura general: cliente–servidor en tres capas

El sistema BioMove adopta una arquitectura cliente–servidor de tres capas (presentación → lógica de negocio → datos), debido a la necesidad de separar claramente las responsabilidades del sistema y garantizar un procesamiento eficiente.

El análisis biomecánico basado en visión artificial requiere un alto consumo de recursos computacionales y el uso de bibliotecas especializadas que no pueden ejecutarse de manera eficiente en dispositivos móviles. Esta limitación técnica obliga a trasladar el procesamiento hacia un entorno de servidor, haciendo que la arquitectura cliente–servidor sea la alternativa más adecuada.

Asimismo, esta arquitectura permite una escalabilidad diferenciada, en la cual la capa de presentación puede evolucionar en términos de funcionalidades e interfaz sin afectar la lógica de negocio, mientras que el backend puede escalar de forma independiente para atender la demanda de procesamiento.

- **Patrón MVVM en el cliente**

En la capa de presentación se adopta el patrón Modelo–Vista–VistaModelo (MVVM), debido a su capacidad para separar la lógica de la interfaz de usuario del manejo del estado y la lógica de negocio.

Este patrón permite una comunicación reactiva entre la vista y los datos, facilitando la actualización automática de la interfaz ante cambios en el estado del sistema. Además, promueve una organización clara del código, donde cada componente cumple una función específica.

Otra ventaja relevante es la mejora en la testabilidad, ya que la lógica de negocio se encuentra desacoplada de la interfaz gráfica, permitiendo realizar pruebas unitarias sin depender de componentes visuales, esta separación contribuye al cumplimiento de características de calidad definidas en la norma ISO/IEC 25010, especialmente en términos de aprendibilidad y operabilidad, al facilitar la gestión del flujo de interacción del usuario y la validación de acciones de manera independiente a la interfaz.

- **Repository Pattern**

Se implementa el patrón Repository con el objetivo de desacoplar la lógica de acceso a datos de la lógica de negocio.

Este enfoque permite abstraer las fuentes de datos, evitando que los cambios en los mecanismos de almacenamiento o en los servicios externos afecten directamente a las capas superiores del sistema. De esta manera, se mejora la mantenibilidad y se facilita la evolución del sistema ante posibles cambios tecnológicos.

- **Capa de servicios**

La arquitectura incorpora una capa de servicios que encapsula funcionalidades específicas del sistema, permitiendo una separación clara de responsabilidades.

Cada servicio se encarga de una función particular, como la comunicación con el backend, la gestión de autenticación o la validación de información previa al envío de datos. Este diseño modular facilita la reutilización de componentes y mejora la organización interna del sistema.

Además, esta capa permite realizar validaciones locales antes de interactuar con el servidor, lo que contribuye a una mejor experiencia de usuario al proporcionar retroalimentación inmediata.

Backend: FastAPI, Celery y Redis

El backend del sistema se implementa utilizando un enfoque que permite gestionar múltiples operaciones de manera concurrente, lo cual resulta fundamental en aplicaciones que requieren procesamiento intensivo y manejo de múltiples solicitudes.

Para el procesamiento de tareas de alta carga, se utiliza un sistema de colas que permite ejecutar procesos en segundo plano sin afectar la disponibilidad del sistema. Este enfoque posibilita la priorización de tareas críticas frente a aquellas que requieren mayor tiempo de ejecución, evitando bloqueos y mejorando el rendimiento general.

Adicionalmente, se emplea un mecanismo de mensajería que permite una comunicación eficiente entre los componentes del sistema, garantizando una baja latencia en la gestión de tareas.